

TP 8

JS Arena # 4 : Implémentation de la classe GameController

Objectifs :

- Implémenter la classe **GameController** pour JS Arena
- Créer un lien entre vos modèles de données (classes **Game** et **Player**) et le backend en communiquant avec le serveur à travers un WebSocket.
- Comprendre le rôle du **contrôleur** dans une architecture client temps réel basée sur le modèle MVC.
- Mettre en place :
 - La boucle de jeu principale qui se répétera tant que votre joueur sera in game.
 - La gestion des entrées clavier
 - La communication WebSocket avec le serveur

Prérequis :

- Vous copierez vos fichiers **Game.js** et **Player.js**, depuis votre dossier **TP7**, et les rangerez dans un dossier **TP8** situé à la racine de votre repo github.
- Vous y copierez également les fichiers **portal.js**, **portal.html** et **portal.css**, ainsi que le dossier **assets/** depuis votre dossier **TP6**

- Vous y créez un fichier **game.html** qui formera votre page principale de jeu, depuis laquelle vous appellerez vos scripts **Player.js**, **Game.js** et **GameController.js**.
- Vous modifierez **portal.js** pour rediriger l'utilisateur sur cette page principale de jeu après soumission du formulaire sur la page portail.
- Vous téléchargerez dans votre dossier **TP8** :
 - Le fichier **GameController.js** fourni à côté du sujet pour y implémenter votre classe **GameController**.
 - le dossier **backend/** fourni à côté du sujet pour pouvoir lancer le backend et communiquer avec en local.

Consignes générales :

- Travailler dans le fichier **GameController.js**
- Utiliser `console.log()` pour afficher les résultats de vos tests
- Installer les dépendances nécessaires pour faire tourner le backend dans un terminal avec la commande suivante :
python -m pip install fastapi uvicorn

À ce stade, **aucun rendu graphique n'est attendu**.

Votre programme fonctionnera uniquement via :

- Les échanges réseaux avec votre backend
- Les logs console

Partie 1 – Implémentation du constructeur de la classe GameController :

Vous effectuerez les tâches suivantes dans le constructeur de la classe **GameController** :

- Créer un attribut contenant une instance de la classe **Game** pour y stocker les informations de la partie et des joueurs.
- Créer des attributs qui prendront les valeurs stockées dans le **localStorage** au moment de la soumission du formulaire de la page portail (name, serverUrl et spritePath).
- Créer un attribut contenant un objet déclaré manuellement avec les propriétés booléennes suivantes :
 - **up**
 - **down**
 - **left**
 - **right**
 - **attack**

Cet attribut vous servira à gérer les événements clavier en gardant un état des lieux clair de vos touches de déplacements et d'attaque. Toutes ses propriétés doivent valoir **false** au moment de la déclaration, afin d'éviter une action non désirée au lancement.

Partie 2 – Connexion au backend :

Nous avons vu dans la schématisation du projet que le frontend et le backend devaient communiquer à intervalles réguliers pour que le jeu fonctionne correctement :

- Le backend doit envoyer des snapshots de l'état de la partie au frontend pour que ce dernier dispose de données récentes pour son affichage.
- Le frontend doit quant à lui envoyer un état des lieux des touches pressées par le client au backend, pour que ce dernier applique les actions correspondantes (déplacements et attaques) avant d'en renvoyer les résultats dans son prochain snapshot.

Pour assurer une bonne réactivité des contrôles et des animations à l'utilisateur (vous), il faut que ces échanges se produisent suffisamment souvent ; raison pour laquelle le serveur renvoie 20 de ces snapshots par seconde (on parle d'une fréquence de rafraîchissement des données de 20Hz), résultant en un temps de réponse de 50ms.

Cette fréquence de rafraîchissement est le fruit d'un compromis étudié pour éviter une surcharge du serveur central (trop de requêtes à traiter), tout en gardant une bonne réactivité côté front (assez d'updates pour garder un temps de réponse bas, donc peu de latence).

Nous utiliserons un [flux TCP](#) pour communiquer avec le backend. Ce protocole, contrairement au protocole [HTTP](#) que l'on utilise pour des API classiques, permet de garder une

connexion ouverte entre deux machines et de transmettre de larges volumes de données rapidement avec très peu de délai, ce qui est idéal pour un jeu en ligne à faible latence.

Il est assez simple de communiquer via TCP en JS grâce au protocole WebSocket, qui est une surcouche du protocole TCP, et dont l'API est directement disponible en JS via la classe [WebSocket](#).

Vous déclarerez un attribut **socket** dans le constructeur de votre classe **GameController**. Vous l'assignerez en tant qu'instance de la classe **WebSocket**. Le constructeur de la classe **WebSocket** prend en paramètre l'URL à laquelle vous souhaitez vous connecter, qui correspondra à l'URL serveur que vous saisirez sur la page portal et enregistré via **localStorage**.

Vous vous servirez de cet attribut **socket** pour gérer l'envoi et la réception de messages dans les parties suivante.

Vous déclarerez pour finir une méthode **initSocket()** dans votre classe **GameController** dans laquelle vous implémenterez les parties 3 et 4.

Partie 3 – Envoi du premier message au backend :

Avant de pouvoir interagir avec le backend, vous devrez le démarrer en lançant la commande suivante depuis le dossier **backend/** dans un terminal :

```
python -m uvicorn main:app --host 0.0.0.0 --port 8000  
--reload
```

L'URL à renseigner sur votre page portail devra alors être la suivante : **ws://localhost:8000/ws**

Afin de recevoir des messages contenant l'état global de la partie depuis le backend, vous devrez d'abord vous connecter à ce dernier en déclarant votre attribut **socket**.

Vous implémenterez ensuite un premier callback et l'assignerez à la propriété **onopen** de votre attribut **socket**.

Ce callback sera appelé au moment de la connexion ; il se lancera en réaction à l'ouverture réussie d'une connexion WebSocket entre votre frontend et le backend (d'où le nom de la propriété **onopen**).

Il sera utilisé dans votre script afin d'envoyer un premier message dit d'*identification*, permettant au backend de vous ajouter à sa liste de joueurs. Pour ce faire, le serveur utilisera les champs **name** et **skinPath** dans ce premier message.

Les protocoles TCP / HTTP permettent avant tout de transmettre du texte brut pour des raisons de polyvalence, et

communiquent la plupart du temps grâce au format de données JSON. Aussi, vous enverrez votre premier message de registration grâce à la méthode **send** de votre socket de la manière suivante :

```
initSocket() {  
  // Triggered when the connection is successfully opened  
  this.socket.onopen = () => {  
    console.log("Connected to server");  
  
    // Send player identity to the server  
    this.socket.send(JSON.stringify({  
      name: this.pseudo,  
      skinPath: this.skinPath  
    }));  
  };  
};
```

La méthode **stringify()** de l'objet **JSON** s'occupe ici de convertir un objet en son équivalent au format JSON dans une chaîne de caractères, permettant ainsi de l'envoyer par message. (Les protocoles HTTP et TCP ne peuvent transmettre que des séries d'octets, aussi appelées "texte", raison pour laquelle on ne peut pas directement envoyer un objet JS par message sans l'avoir converti en texte au préalable, en l'occurrence au format JSON car c'est le format imposé par notre backend).

Partie 4 – Réception de messages envoyés par le backend :

Il vous faudra également implémenter un callback vous permettant de réagir à la réception d'un message envoyé par le backend. Vous assignerez ce callback à la propriété **onmessage** de votre attribut de classe **socket**, ce qui aura pour effet de l'exécuter à chaque fois que vous recevrez un message du backend. Vous utiliserez ce callback pour :

- Transformer les données reçus sous forme de chaîne de caractères au format JSON en objet JS.
- Utiliser l'objet JS résultant pour mettre à jour les données stockées dans votre modèle.

Partie 5 – Gestion des événements clavier :

Dans cette partie, vous allez mettre en place la gestion des entrées clavier du joueur.

L'objectif n'est pas de déplacer directement un joueur à l'écran, mais de maintenir à jour l'état des touches pressées afin de pouvoir l'envoyer régulièrement au backend.

Plutôt que de réagir directement à chaque appui de touche en envoyant un message, nous allons plutôt conserver un état global des touches dans l'attribut **inputState** que vous avez déclaré dans votre constructeur.

Vous créerez une méthode ***initInput()*** dans la classe **GameController**, qui attachera deux écouteurs d'événements :

- Le premier se chargera de gérer les event correspondants à des pressions sur les touches (***keyup***); son rôle sera de passer les propriétés correspondants aux touche(s) pressée(s) à **true** dans votre objet **inputState**.
- Le second se chargera de gérer les event correspondants à des relâchements de touches (***keydown***) ; son rôle sera

de passer les propriétés correspondants aux touche(s) relâchée(s) à **false** dans votre objet **inputState**.

Vous appellerez la méthode ***initInput()*** depuis le constructeur de **GameController**, afin que les écouteurs d'événements soient attachés dès le lancement de la page game.

Astuce : Vous pouvez utiliser un **console.log(this.inputState)** dans vos callbacks pour vérifier que l'état des touches change correctement lorsque vous appuyez et relâchez les touches.

Partie 6 – Envoi de messages au backend :

Maintenant que l'état des touches est correctement suivi côté frontend, il faut transmettre ces informations au backend afin qu'il puisse appliquer les actions correspondantes (déplacements, attaques...).

Le backend fonctionne avec une fréquence de 20 mises à jour par seconde (20 Hz). Pour rester synchronisé avec lui et garder une expérience fluide, le frontend doit envoyer l'état des touches à la même fréquence.

Nous allons donc envoyer l'objet **inputState** au backend via la connexion WebSocket 20 fois par seconde.

Vous créerez pour ce faire une méthode ***startInputSender()*** dans votre classe **GameController**.

Dans cette méthode, vous utiliserez la Web API ***setInterval()*** afin d'appeler un callback toutes les **this.SERVER_INTERVAL** millisecondes.

Vous vous inspirerez de la partie 3 pour implémenter ce callback, qui effectuera les tâches suivantes :

- Vérifier que la connexion WebSocket est bien ouverte, ni occupée ni fermée (**WebSocket.OPEN**), et arrêter l'exécution du callback si ce n'est pas le cas.
- Envoyer un message au backend contenant :
 - un champ **type** avec la valeur "input"
 - un champ **input** contenant l'objet **inputState**

Vous appellerez la méthode **startInputSender()** depuis le constructeur de **GameController** après l'appel à **initSocket()**, ce qui lancera l'envoi de messages réguliers au backend dès le lancement du script.

Partie 7 – Boucle de rendu :

Dans cette partie, vous testerez le fonctionnement de votre communication frontend-backend grâce à la **boucle principale** du jeu implémentée via la fonction **loop**.

Même si aucun rendu graphique n'est encore réalisé, cette boucle sera bientôt responsable d'afficher chaque image du jeu sur votre page game.

En JavaScript, la fonction ***requestAnimationFrame*** permet d'exécuter une fonction avant chaque rafraîchissement de l'écran, généralement à 60 images par seconde.

Pour créer une boucle, la fonction appelée par ***requestAnimationFrame*** doit s'appeler elle-même à la fin de son exécution.

Vous vous servirez de cette boucle pour afficher les informations censées être mises à jour par le backend suite aux actions que vous effectuerez via votre clavier.

